# *An Introduction to Relational Databases*

# What will you learn ?

1. Data model
2. Relational Model Concepts
3. An Informal Look at the Relational Model
4. Relvars
5. Optimization
6. Data Dictionary
7. Base Relvars and Views
8. Transactions

# 1. Data model

A data model is an abstract, self-contained, logical definition of the objects, operators.

Objects: model the structure of data (e.g., data type, relationships, and constraints).

Operators : model the behavior of objects.

A database schema is the description of a database using a data model.

Data modeling is the process of creating a data model for an application.

# Categories of Data Models

Based on level of data abstraction

- Conceptual level (high-level)
  - Provide concepts that are close to the way many users perceive data.
  - Approaches: Entity-relationship (ER) model, semantic object model, etc.

    Semantic Object Model: Define semantic objects; Use semantic object diagrams to build data models;
- Logical level (DBMS level)
  - Provide concepts that are close to the way data is organized but still may be understood by end users.
  - Approaches: relational, object-oriented, network, and hierarchical model.
- Physical level (low-level)
  - Provide concepts that describe the details of how data is stored in the computer.

# 2. Relational Model Concepts

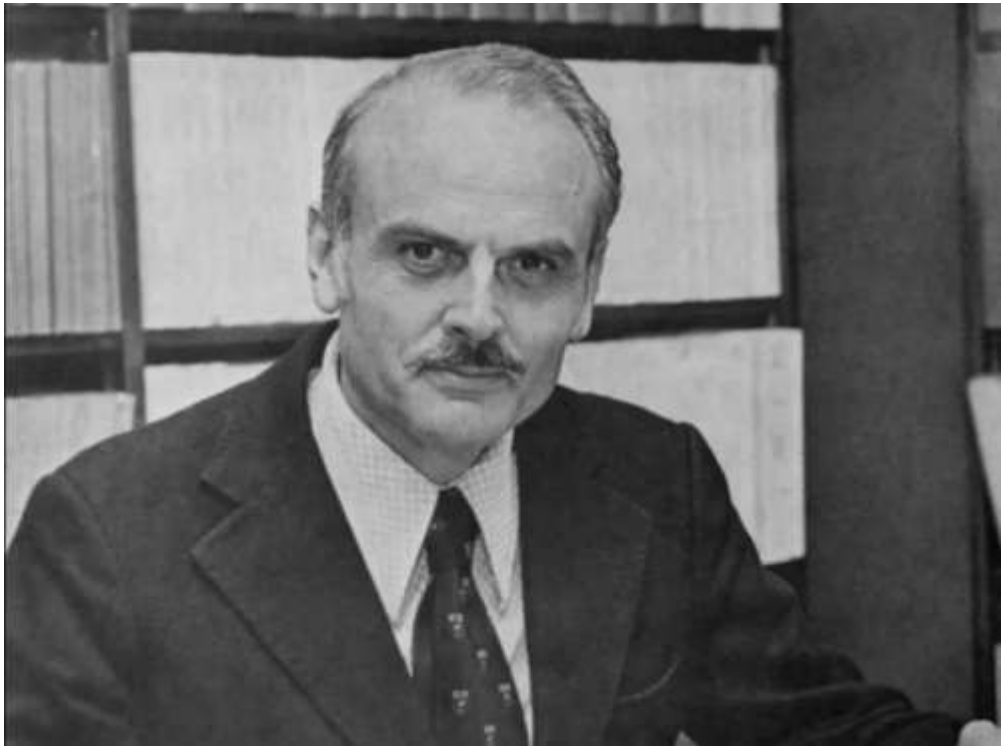
Relational data model: represents the data in a database as a collection of relations (tables).

- First introduced by Codd in 1970
- Based on mathematical notion of domain and relation
- A relation can be easily represented in a table format although they are not the same.

1981年图灵奖获得者：
埃德加·科德
(Edgar Frank Codd)
—“关系数据库之父”

- Relational data model is just one data modeling technique for a database.
- Relational systems are based on the relational model.

## 在IBM加州圣何塞硅谷实验室里工作的数学家兼计算机科学家 Edgar Frank Codd---数据库产业的祖师爷



英格兰人，二战时候是飞行员，后来到美国给IBM服务。再后来，因为美国麦卡锡风潮辗转去了加拿大。之后又回美国IBM工作，顺便去密西根大学拿了一个PhD。Edgar Codd的PhD做的是冯诺依曼架构计算模型的扩展，非常的理论。

1970年，在加州圣何塞硅谷实验室里工作的Edgar Frank Codd公开发表了一篇论文：A Relational Model of Data for Large Shared Data Banks。翻译成中文就是一个为大容量共享数据银行设计的数据的关系模型。提出了数据的关系模型，也就是著名的关系代数。

这篇论文是基于1969年他在IBM的内部工作报告的基础上修改的。所以关系模型到底诞生于哪一年，是一个有争论的事情。很多人通常认为1970年是它诞生的时候。

从此以后，他就开启了关系数据库长达50多年至今屹立不倒的整个产业。他1981年获得计算机界最高奖图灵奖，2003年年去世。2004年为了他，SIGMOD把其最高奖改名为SIGMOD Edgar F. Codd Innovations Award。

SIGMOD,全称数据管理国际会议(Special Interest Group on Management Of Data),是由美国计算机协会(ACM)数据管理专业委员会(SIGMOD)发起、在数据库领域具有最高学术地位的国际性学术会议

# Relational Model Concepts(Continued)

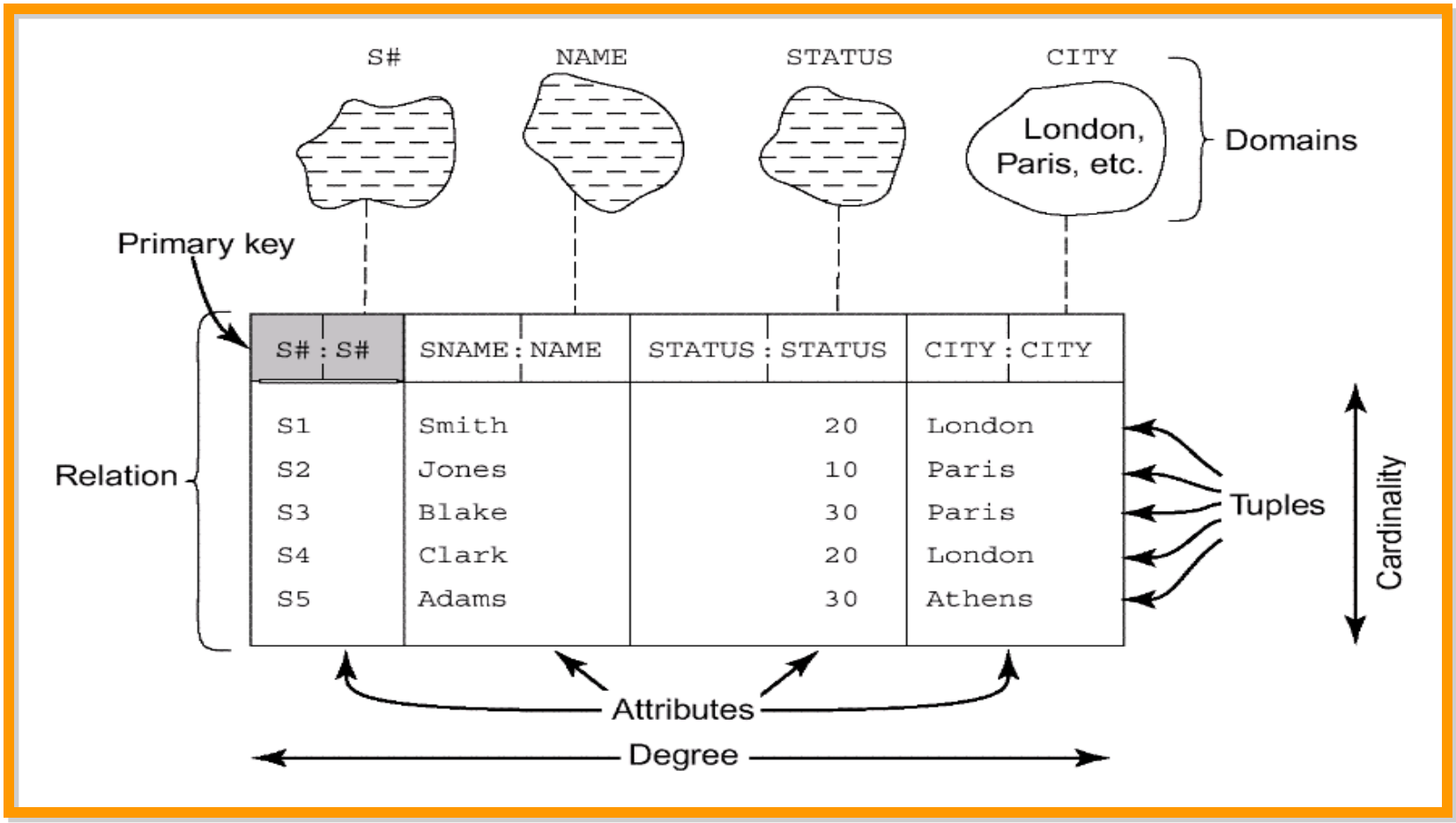



**Fig. 5.1 Structural terminology**

# Relational Model Concepts(Continued)

Attribute (field or column)

Relation (table)

e.g., person, emp ,course, etc.

Tuple (record or row)

The set of attribute values for an object.

Cardinality

The number of tuples

# Relational Model Concepts(Continued)

## Degree

The number of attributes

## Domain (type with some constraints)

The set of all possible values an attribute can have.

It is a description of the format (type, length, etc.) and the semantics of an attribute.

Domains are data types

Domains are object classes

# Relational Model Concepts(Continued)

S

| S# | SNAME | STATUS | CITY |
|---|---|---|---|
| S1 | Smith | 20 | London |
| S2 | Jones | 10 | Paris |
| S3 | Blake | 30 | Paris |
| S4 | Clark | 20 | London |
| S5 | Adams | 30 | Athens |

SP

| S# | P# | QTY |
|---|---|---|
| S1 | P1 | 300 |
| S1 | P2 | 200 |
| S1 | P3 | 400 |
| S1 | P4 | 200 |
| S1 | P5 | 100 |
| S1 | P6 | 100 |
| S2 | P1 | 300 |
| S2 | P2 | 400 |
| S3 | P2 | 200 |
| S4 | P2 | 200 |
| S4 | P4 | 300 |
| S4 | P5 | 400 |

P

| P# | PNAME | COLOR | WEIGHT | CITY |
|---|---|---|---|---|
| P1 | Nut | Red | 12.0 | London |
| P2 | Bolt | Green | 17.0 | Paris |
| P3 | Screw | Blue | 17.0 | Rome |
| P4 | Screw | Red | 14.0 | London |
| P5 | Cam | Blue | 12.0 | Paris |
| P6 | Cog | Red | 19.0 | London |

Fig. 3.8. The Suppliers and parts database(sample values)

# Relational Model Concepts(Continued)

## Primary key

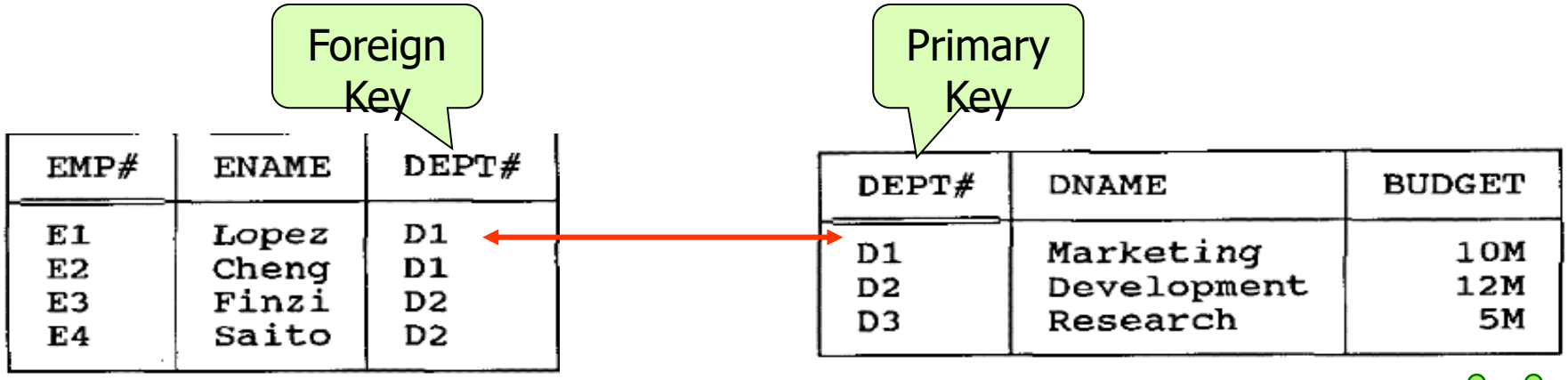
is an attribute, or a collection of attributes, whose values uniquely identify each tuple in a relation.

## Relationship

is the related information between relations. Relationship can be represented by common attribute (s) between relations.

## Foreign key

is the common attribute (s) between relations and that is a primary key in other related relation.



# Relational Model Concepts(Continued)

## A precise definition of the term relation

Given a collection of n types or domains Ti (i = 1, 2, ..., n), not necessarily all distinct, r is a relation on those types if it consists of two parts, a heading and a body, where:

- The heading is a set of n attributes of the form Ai:Ti, where the Ai (which must all be distinct) are the attribute names of r and the Ti are the corresponding type names (i = 1, 2, ..., n);
- The body is a set of m tuples t, where t in turn is a set of components of the form Ai:vi in which vi is a value of type Ti—the attribute value for attribute Ai of tuple t (i = 1, 2, ..., n).

The values m and n are called the cardinality and the degree, respectively, of relation r.

Table=relation?

# Properties of relations

1 There are no duplicate tuples; Column names must be unique;

| ENGINE_1 | ENGINE_2 | ENGINE_3 | ENGINE_4 |
|---|---|---|---|

Column names must be unique

AIRCRAFT

| SERIAL_NUMBER | ACQUIRED | ENGINE | ENGINE | ENGINE | ENGINE |
|---|---|---|---|---|---|
| B238725737 | 1994-07-21 | P0102313 | P0102314 | | |
| B238768737 | 1997-05-12 | R0942497 | R0942498 | | |
| B167029747 | 1992-10-20 | G0015237 | G0015240 | G0025635 | G0025678 |
| A11599320 | 1994-02-19 | R0307023 | R0307025 | | |
| A11599320 | 1994-02-19 | R0307023 | R0307025 | | |
| A203623340 | 1996-08-01 | R0346723 | R0346724 | R0346743 | R0346744 |

Rows should be unique

# Properties of relations

2 Tuples are unordered, top to bottom;

3 Attributes are unordered, left to right;

| TYPE | MODEL | CATEGORY | MANUFACTURER | ENGINES |
|---|---|---|---|---|
| A340 | 100 | JET | AIRBUS | 4 |
| B737 | 500 | JET | BOEING | 2 |
| B737 | 700 | JET | BOEING | 2 |
| A320 | 200 | | | |
| B747 | 400 | | | |

First time

**Unordered Retrieval**

- Next time, *rows* may be returned in a **different sequence**
- Next ti

Next time

| TYPE | MODEL | CATEGORY | ENGINES | MANUFACTURER |
|---|---|---|---|---|
| A320 | 200 | JET | 2 | AIRBUS |
| A340 | 100 | JET | 4 | AIRBUS |
| B737 | 500 | JET | 2 | BOEING |
| B737 | 700 | JET | 2 | BOEING |
| B747 | 400 | JET | 4 | BOEING |

# Properties of relations

4 Each tuple contains exactly one value for each attribute.

☞ A relation that satisfies this fourth property is said to be normalized, or equivalently to be in first normal form.

# 3. An Informal Look at the Relational Model

Structural aspect: tables

Integrity aspect: Those tables satisfy certain integrity constraints (IC)

Manipulative aspect: operators

restrict, project, and join.

# Structural aspect: Table

DEPT

| DEPT# | DNAME | BUDGET |
|---|---|---|
| D1 | Marketing | 10M |
| D2 | Development | 12M |
| D3 | Research | 5M |

EMP

| EMP# | ENAME | DEPT# | SALARY |
|---|---|---|---|
| E1 | Lopez | D1 | 40K |
| E2 | Cheng | D1 | 42K |
| E3 | Finzi | D2 | 30K |
| E4 | Saito | D2 | 35K |

**Fig. 3.1 The dept and emp relation in a table format**

# Structural aspect: Table (Continued)

Tables are the logical structure in a relational system, not the physical structure.

At the physical level, in fact, the system is free to store the data any way it likes—using sequential files, indexing, hashing, pointer chains, compression, etc.

The Information Principle: The entire information content of the database is represented in one and only one way, namely as explicit values in column positions in rows in tables. There are no pointers connecting one table to another.

# integrity constraints

Primary Key constraints

- Primary key value must be unique for every tuple in a relation
- No primary key value can be NULL

Domain constraints

- The constraints on the domain of attributes.
- (CHECK clause)

Referential integrity constraint

- Foreign key constraint, e.g., A tuple in one relation that refers to another relation must refer to an existing tuple in that relation.
- To maintain the consistency among tuples of the two related relations.

Other constraints: functional dependency, assertions, triggers.

# operators

## The property of relational operators

Closure property:The result of relational operations is table(set at a time).

Nested expressions:The output from one operation can become input to another.

Relational languages are nonprocedural, on the grounds that users specify what, not how—i.e., they say what they want, without specifying a procedure for getting it. (automatic navigation)

```
INSERT INTO SP ( S#, P#, QTY )
       VALUES ( 'S4', 'P3', 1000 ) ;
```

```
MOVE 'S4' TO S# IN S
FIND CALC S
ACCEPT S-SP-ADDR FROM S-SP CURRENCY
FIND LAST SP WITHIN S-SP
while SP found PERFORM
    ACCEPT S-SP-ADDR FROM S-SP CURRENCY
    FIND OWNER WITHIN P-SP
    GET P
    IF P# IN P < 'P3'
       leave loop
    END-IF
    FIND PRIOR SP WITHIN S-SP
END-PERFORM
MOVE 'P3' TO P# IN P
FIND CALC P
ACCEPT P-SP-ADDR FROM P-SP CURRENCY
FIND LAST SP WITHIN P-SP
while SP found PERFORM
    ACCEPT P-SP-ADDR FROM P-SP CURRENCY
    FIND OWNER WITHIN S-SP
    GET S
    IF S# IN S < 'S4'
       leave loop
    END-IF
    FIND PRIOR SP WITHIN P-SP
END-PERFORM
MOVE 1000 TO QTY IN SP
FIND DB-KEY IS S-SP-ADDR
FIND DB-KEY IS P-SP-ADDR
STORE SP
CONNECT SP TO S-SP
CONNECT SP TO P-SP
```

Fig 3.5 Automatic vs. manual navigation

# 4. Relvars

Relvar:relational variables

E.g. Customer,Product,Ptype,DEPT,EMP

The values of Relvar is relation values.

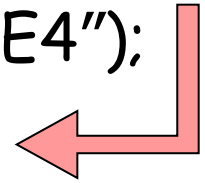
The relation values is different at different times.

delete →relational assignment operation

# Relations and Relvars (Continued)

## Relational assignment

EMP:=EMP MINUS (EMP WHERE E#="E4");



DELETE EMP WHERE E#="E4";

EMP

| EMP# | ENAME | DEPT# | SALARY |
|---|---|---|---|
| E1 | Lopez | D1 | 40K |
| E2 | Cheng | D1 | 42K |
| E3 | Finzi | D2 | 30K |

# 5. Optimization

Relational systems are sometimes said to perform automatic navigation.

For each relational request from the user, it is the job of the optimizer to choose an efficient way to implement that request.

e.g. RESULT:=(EMP WHERE EMP#='e4'){SALARY};

two ways of performing the necessary data access:

By doing a physical sequential scan of (the stored version of) relvar EMP until the required data is found;

If there is an index on (the stored version of) the EMP# column, use that index and going directly to the required data.

# Optimization (Continued)

The basis of considerations of optimization:

- Which relvars are referenced in the request;
- How big those relvars currently are;
- What indexes exist;
- How selective those indexes are;
- How the data is physically clustered on the disk;
- What relational operations are involved;

# 6. The Catalog (or Data Dictionary)

contains detailed information (sometimes called descriptor information or metadata) regarding the various objects that are of interest to the system itself.

The place where

all of the various schemas (external, conceptual, internal)

all of the corresponding mappings (external/ conceptual, conceptual/internal)

are kept.

Optimizer uses... security subsystem use...

# The Catalog…

The catalog itself consists of system relvars.

TABLE

| TABNAME | COLCOUNT | ROWCOUNT | ..... |
|---|---|---|---|
| DEPT | 3 | 3 | ..... |
| EMP | 4 | 4 | ..... |
| ..... | ..... | ..... | ..... |

COLUMN

| TABNAME | COLNAME | ..... |
|---|---|---|
| DEPT | DEPT# | ..... |
| DEPT | DNAME | ..... |
| DEPT | BUDGET | ..... |
| EMP | EMP# | ..... |
| EMP | ENAME | ..... |
| EMP | DEPT# | ..... |
| EMP | SALARY | ..... |
| ..... | ..... | ..... |

Fig. 3.6 Catalog for the departments and employees database

# The Catalog…

users can interrogate the catalog in exactly the same way they interrogate their own data.

E.g.

1.what columns relvar DEPT contains?

(COLUMN WHERE TABNAME='DEPT'){COLNAME};

2.Which relvars include a column called EMP#?

(COLUMN WHERE COLNAME='EMP#'){TABNAME};

3.What does the following do?

((TABLE JOIN COLUMN)

WHERE COLCOUNT<5){TABNAME,COLNAME};

# 7. Base Relvars and Views

P71~75

The original (given) relvars are called base relvars, and their relation values are called base relations (real relvar);

A relation that is or can be obtained from those base relations by means of some relational expression is called a derived or derivable relation.

# View

A view is a derived relvar(virtual relvar).

e.g. CREATE VIEW TOPEMP AS

(EMP WHERE SALARY >33K){EMP#,ENAME,SALARY};

TOPEMP

| EMP# | ENAME | DEPT# | SALARY |
|---|---|---|---|
| E1 | Lopez | D1 | 40K |
| E2 | Cheng | D1 | 42K |
| E3 | Finzi | D2 | 30K |
| E4 | Saito | D2 | 35K |

Fig. TOPEMP as a view of EMP(unshaded portions)

# View (Continued)

A view can be derived from several tables or views.

E.g.

```
CREATE VIEW JOINEX AS
  ((EMP JOIN DEPT)
     WHERE  BUDGE>7M){EMP#,DEPT#};
```

# View (Continued)

The view is effectively just a window into the underlying base relvar.

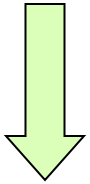
Any changes to that underlying relvar will be automatically and instantaneously visible through that window.

# View (Continued)

Changes to TOPEMP will automatically and instantaneously be applied to relvar EMP.

e.g.

(TOPEMP WHERE SALARY<42k){EMP#,SALARY}

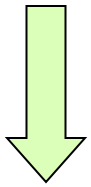


((EMP WHERE SALARY>33K){EMP#,ENAME,SALARY})
WHERE SALARY<42k){EMP#,SALARY}

# View (Continued)

e.g.

DELETE TOPEMP WHERE SALARY<42k;



DELETE EMP
WHERE SALARY>33K AND SALARY<42K;

在SQL语言中，引入视图机制的主要优点：

简化用户的操作；

能以多种角度看待同一数据；

对重构数据库提供了一定程度的逻辑独立性；

对机密数据提供安全保护；

可以更清晰的表达查询。

# 8. Transactions

A transaction is a logical unit of work, typically involving several database operations.

BEGIN TRANSACTION, COMMIT, and ROLLBACK

# An example of transaction

```
BEGIN TRANSACTION
...
/*读出帐户Acct1的余额Balance1*/
EXEC SQL SELECT balance
        INTO :Balance1
        FROM Accounts
        WHERE AccountNo=:Acct1;
/*从帐户Acct1中减去金额Amount*/
EXEC SQL UPDATE Accounts
    SET balance=balance-:Amount
    WHERE AccountNo=:Acct1;
```

```
IF （Balance1<Amount）
{
    printf（"金额不足，不能转帐！"）；
    /*恢复事务，撤消所有操作*/
    EXEC SQL ROLLBACK;
}
ELSE
{
    /*对帐户Acct2增加金额Amount*/
    EXEC SQL UPDATE Accounts
            SET balance=balance+:Amount
            WHERE AccountNo=:Acct2;
    EXEC SQL COMMIT;    /*提交事务*/
} ...
```

# property of transactions

Atomic: Transactions are guaranteed either to execute in their entirety or not to execute at all.

Durability: Once a transaction successfully executes COMMIT, its updates are guaranteed to be applied to the database, even if the system subsequently fails at any point.

# property of transactions (Continued)

Isolation: Database updates made by a given transaction T1 are not made visible to any distinct transaction T2 until and unless T1 successfully executes COMMIT.

Serialization: The interleaved execution of a set of concurrent transactions produces the same result as executing those same transactions one at a time in some unspecified serial order.

# Example

| 事务T1 | 事务T2 | 数据库中的值 |
|---|---|---|
| read (A) | | A=50 |
| A:=A-2 | | |
| write (A) | | A=48 |
| read (B) | | B=100 |
| B:=B-1 | | |
| write (B) | | B=99 |
| | read (A) | A=48 |
| | A:=A-2 | |
| | write (A) | A=46 |
| | read (B) | B=99 |
| | B:=B-1 | |
| | write (B) | B=98 |

串行调度1：先T1后T2

| 事务T1 | 事务T2 | 数据库中的值 |
|---|---|---|
| | read (A) | A=50 |
| | A:=A-2 | |
| | write (A) | A=48 |
| | read (B) | B=100 |
| | B:=B-1 | |
| | write (B) | B=99 |
| read (A) | | A=48 |
| A:=A-2 | | |
| write (A) | | A=46 |
| read (B) | | B=99 |
| B:=B-1 | | |
| write (B) | | B=98 |

串行调度2：先T2后T1

Result:A=46,B=98

# Example…

| 事务T1 | 事务T2 | 数据库中的值 |
|---|---|---|
| read (A) | | A=50 |
| A:=A-2 | | |
| write (A) | | A=48 |
| | read (A) | |
| | A:=A-2 | |
| | write (A) | A=46 |
| read (B) | | B=100 |
| B:=B-1 | | |
| write (B) | | B=99 |
| | read (B) | |
| | B:=B-1 | |
| | write (B) | B=98 |

并发调度1

| 事务T1 | 事务T2 | 数据库中的值 |
|---|---|---|
| read (A) | | A=50 |
| A:=A-2 | | |
| write (A) | | A=48 |
| read (B) | | B=100 |
| | read (A) | A=48 |
| | A:=A-2 | |
| | write (A) | A=46 |
| | read (B) | B=100 |
| B:=B-1 | | |
| write (B) | | B=99 |
| | B:=B-1 | |
| | write (B) | B=98 |

并发调度2

Result:
A=46,B=98 √

Result:
A=46,B=98 √

例： 设有一个飞机机票订票系统，考虑如下售票活动的并发操作问题：

1、售票员A通过网络在数据库中读出某航班的机票余额为y张，设y=15；

2、售票员B通过网络在数据库中读出某航班的机票余额为y张，设y=15；

3、售票员A卖出一张机票，然后修改余额为y=y-1，此时，y=14，将14写进数据库；

4、售票员B卖出一张机票，然后修改余额为y=y-1，此时，y=14，将14写进数据库；

这个售票活动并发操作的结果是：实际已经卖出2张机票，但在数据库中机票余额仅减少1，从而导致错误。这就是著名的飞机机票订票系统问题。

这个问题之所以会产生错误，其根本原因在于两个用户反复交叉地使用同一个数据库的结果。

# 多事务执行方式

-事务串行执行（serial)

每个时刻只有一个事务运行，其他事务必须等到这个事务结束以后方能运行。

- 不能充分利用系统资源，不能发挥数据库共享资源的特点。

-交叉并发方式(interleaved concurrency)

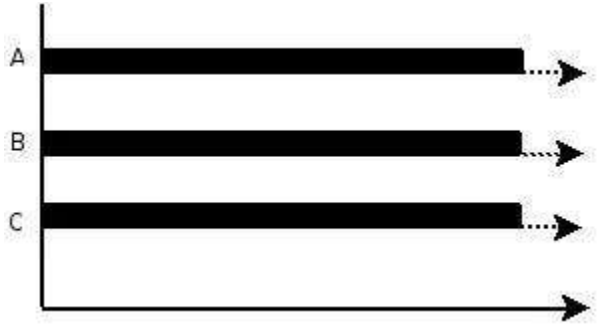
是这些并行事务的并行操作轮流交叉运行，是单处理机系统中的并发方式。

- 能够减少处理机的空闲时间，提高系统的效率。

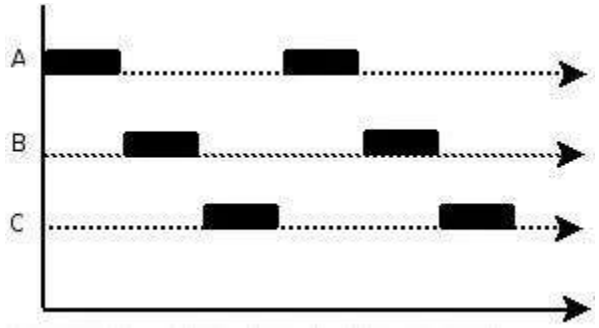
-同时并发方式(simultaneous concurrency>

多处理机系统中，每个处理机可以运行一个事务，多个处理机可以同时运行多个事务，实现多个事务真正的并行运行。

- 最理想的并发方式，但受制于硬件环境。




并行(parallel)：指在同一时刻，有多条指令在多个处理器上同时执行。所以无论从微观还是从宏观来看，二者都是一起执行的。




并发(concurrency)：指在同一时刻只能有一条指令执行，但多个进程指令被快速的轮换执行，使得在宏观上具有多个进程同时执行的效果，但在微观上并不是同时执行的，只是把时间分成若干段，使多个进程快速交替的执行。

并行在多处理器系统中存在，而并发可以在单处理器和多处理器系统中都存在，并发能够在单处理器系统中存在是因为并发是并行的假象。并行要求程序能够同时执行多个操作，而并发只是要求程序假装同时执行多个操作（每个小时间片执行一个操作，多个操作快速切换执行）。

在数据库管理系统中对事务采用并发机制的主要目的有两个：

1、改善系统的资源利用率

对于一个事务来讲，在不同的执行阶段需要的资源不同，有时需要CPU，有时需要访问磁盘，有时需要I/O、有时需要通信。如果事务串行执行，有些资源可能会空闲；如果事务并发执行，则可以交叉利用这些资源，有利于提高系统资源的利用率。

2、改善短事务的响应时间

设有两个事务T1 和T2，其中T1是长事务，交付系统在先；T2是短事务，交付系统比T1稍后。如果串行执行，则须等T1执行完毕后才能执行T2。而T2的响应时间会很长。一个长事务的响应时间长一些还可以得到用户的理解，而一个短事务的响应时间过长，用户一般难以接受。

如果T1 和T2并发执行，则T2可以和T1重叠执行，可以较快地结束，明显地改善其响应时间。